**Task 1 – System Understanding Note**

**1. What is the main purpose of the application?**

Campus Supply Desk is a Python application that manages orders for a campus supply store. Customers can view products, request quotes, place orders, review order history, retrieve specific orders, and cancel purchases. The system checks available inventory before accepting an order and adjusts stock when an order is placed or cancelled.

Prices depend on the selected products, quantities, customer type, and fulfillment method. Students receive a 10% discount when the subtotal reaches $50. Delivery costs $6 unless the amount after discount reaches $100. The application stores inventory and order records in JSON files so information remains available after restarting.

**2. What are the major classes, modules, or components, and what does each do?**

The repository contains several components:

- **service.py:** Handles business operations, including quotes, order placement, pricing, stock checks, and cancellation.
- **storage.py:** Manages saving and loading application data using JSON files.
- **menu.py:** Supports the interactive terminal interface and displays catalog and order information.
- **cli.py:** Supports individual command-line operations.
- **__main__.py:** Provides the entry point for running python -m supplydesk.
- **catalog.json:** Supplies the initial product catalog, including product IDs, prices, and quantities.
- **test_smoke.py:** Contains basic automated tests for checking application behavior.

These components separate user interaction, order processing, and data storage.

**3. Identify one important execution flow and explain how the components work together.**

Placing an order is an important execution flow. The customer selects products, enters quantities, and chooses a customer type and fulfillment method through the menu or command line. The request reaches the service layer, which must check the product IDs, quantities, customer type, fulfillment option, and available stock.

After validation, the service calculates line subtotals, any student discount, delivery charges, and the final total. A successful placement receives an order ID beginning with ORD-0001 and the status placed. Stock quantities decrease, and the order and updated inventory are saved. In this expected flow, **menu.py or cli.py handles input, service.py handles ordering rules, and storage.py handles persistence**.

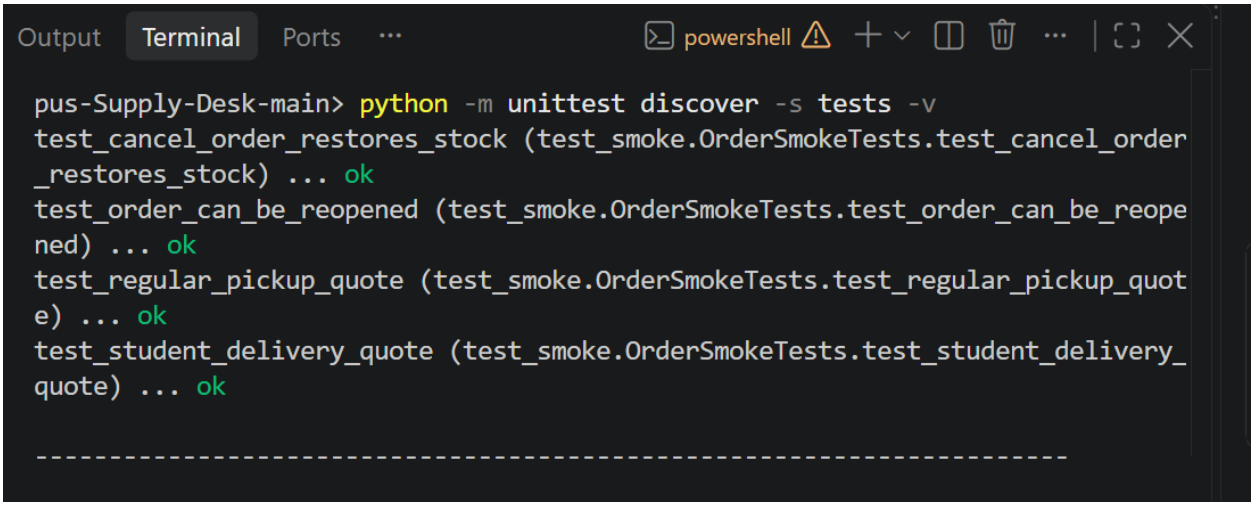
**4. Which part of the code appears most difficult to understand or maintain, and why?**

The order-processing logic in **service.py** appears to be the area most likely to require careful maintenance because several rules interact. Validation must finish before stock changes, duplicate product lines must be considered together, and quotes must not modify saved information. Pricing also depends on discount and delivery thresholds.

Keeping these responsibilities clearly separated would make changes easier to test and reduce the risk of incorrect totals or inventory updates. In the original service.py, quote() and place_order() repeated validation and pricing logic, which made changes harder to maintain consistently.

Task 2

1.




Baseline record for Task 2: 4 tests passed, 0 failures, 0 errors.

**Task 2 – Build a Testing Safety Net**

Before refactoring, I ran the four existing tests and added seven automated tests using Python's unittest framework. The tests were designed to check normal behavior, boundary conditions, invalid inputs, and functionality that should remain unchanged during refactoring.

**1. Normal Behavior**

I tested placing orders with multiple products and generating sequential order IDs. These tests are important because the application must process valid orders correctly and maintain accurate order records.

**Expected Result:** Valid orders should be processed successfully, and order IDs should increase in sequence.

**Actual Result:** Both tests passed.

**2. Boundary Behavior**

I tested the student discount at exactly $50 and free delivery at the required threshold. These tests verify whether the application handles pricing limits correctly.

**Expected Result:** A $50 student order should receive a $5 discount, and an eligible order at the free-delivery threshold should have no delivery charge.

**Actual Result:** The free-delivery test passed, but the student-discount test failed. The application returned a discount of 0 cents instead of 500 cents.

**3. Invalid or Exceptional Behavior**

I tested an unsupported customer type and a Boolean quantity. These tests matter because invalid inputs should be rejected rather than processed as valid orders.

**Expected Result:** Both requests should raise ValueError.

**Actual Result:** Both tests passed.

**4. Behavior Important to Preserve**

I tested whether an invalid order leaves a previously successful order unchanged. I also retained the original tests covering quotes, order persistence, and cancellation.

**Expected Result:** Previous orders should remain saved, quotes should calculate correctly, and cancellation should restore stock.

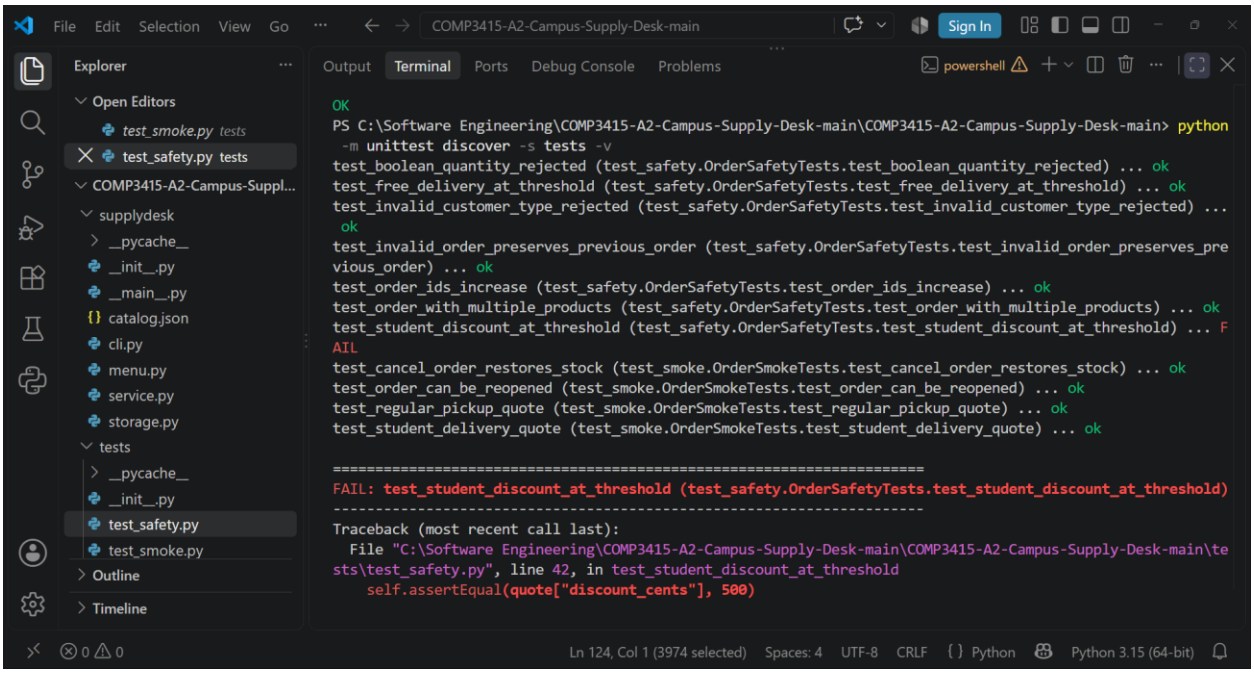
**Actual Result:** All tests in this category passed.

**Baseline Test Results**

- Original tests: 4 passed.
- Additional tests: 6 passed, 1 failed.
- Total: 11 tests executed, 10 passed, 1 failed, 0 errors.
- Execution time: 0.128 seconds.

**Identified Defect:** The student discount is not applied when the subtotal equals exactly $50, even though the requirements specify that the discount applies at $50 or above.

The failing test was retained without changing its expected result. This failure will be investigated during the code analysis and addressed as appropriate during refactoring.




**Task 3**

| Code Smell | Location | Problem | Evidence | Consequence | Proposed Improvement |
|---|---|---|---|---|---|
| **Duplicated Code** | supplydesk/service.py – OrderService.quote() and OrderService.place_order() | Both methods contain nearly identical code for validating customer information, checking products and quantities, | Both methods contain nearly identical code for validating customer information, checking products and quantities, | Repeating the same logic makes maintenance difficult because changes must be made in multiple locations. If one | Extract the shared validation and pricing calculations into reusable helper methods. Both quote() and place_order() |

| | | | | | |
|---|---|---|---|---|---|
| | | calculating subtotals, and applying discounts and delivery charges. | calculating subtotals, and applying discounts and delivery charges. | method is updated and the other is missed, quotes and placed orders could produce inconsistent results. | should use these methods to ensure consistent behavior |
| **Long Method and Multiple Responsibilities** | supplydesk/service.py – OrderService.place_order() | The place_order() method handles too many responsibilities within a single method. | It validates customer information, searches for products, checks stock, calculates prices, creates the order dictionary, deducts inventory, increases the order number, and saves the updated state using self.store.save(state). | Combining these operations makes the method harder to understand, debug, and modify. A change to one responsibility, such as inventory management, could unintentionally affect pricing or order creation. | Break the method into smaller functions with clear responsibilities, such as validating requests, preparing order details, and updating inventory. This would improve readability and make individual operations easier to test. |
| **Repeated Product Searching** | supplydesk/service.py – OrderService.quote(), OrderService.place_order(), and OrderService.cancel_order() | **:** The application repeatedly searches through the entire product catalog to find products using their IDs. | The methods use loops such as for candidate in state["catalog"] and compare candidate["product_id"] with the requested product ID. In place_order(), products are searched again when stock is deducted, even though they were already located during validation. | Repeated catalog searches make the code unnecessarily complex and increase processing time as the number of products grows. The same lookup logic is also scattered across different operations. | Create a dictionary that maps product IDs to product records. This would allow direct product lookup and reduce repeated loops, making the code simpler and easier to maintain |
| **Missing Test Coverage for Combined Stock Quantities** | tests/test_smoke.py and supplydesk/service.py – OrderService.quote() and OrderService.place_order() | The original test suite does not check whether the application correctly handles duplicate product lines whose combined quantities exceed available stock. | The original test_smoke.py contains four tests covering pricing, order persistence, and cancellation. None tests duplicate product lines. In service.py, the condition if quantity > product["stock"] checks each line individually rather than the total | An order containing three canvas bags on one line and two on another could pass validation even though only four bags are available. This could result in negative inventory after placement. Without a test for this situation, the defect could remain unnoticed or | Add automated tests that submit duplicate product lines exceeding available stock. The expected result should be a ValueError, with no changes to inventory, order history, or order numbering. Update validation to check the combined quantity for each |

| | | | | | |
|---|---|---|---|---|---|
| | | | requested quantity for a product. | reappear after refactoring. | product before processing the order. |
| **Incorrect Student Discount Boundary** | supplydesk/service.py – OrderService.quote() and OrderService.place_order() | The student discount condition incorrectly excludes orders with a subtotal of exactly $50. | Both methods use if customer_type == "student" and subtotal > 5000. However, the functional requirements specify that students receive a 10% discount when the subtotal is **at least 5,000 cents**. During baseline testing, test_student_discount_at_threshold failed because the expected discount was 500 cents, but the actual result was 0 cents. | Students placing orders worth exactly $50 are charged more than required. The incorrect condition affects pricing accuracy in both quotes and placed orders | Change the comparison from subtotal > 5000 to subtotal >= 5000 in the shared pricing logic. Keep the existing boundary test to verify that the discount is correctly applied after the change. |

**Task 4**

**Refactoring 1: Duplicated Code**

**Before:** The original quote() and place_order() methods repeated the same validation and pricing logic. Both checked customer types, fulfillment methods, product IDs, quantities, stock availability, discounts, and delivery charges.

**Refactoring:** I extracted the shared validation and pricing logic into a helper method called _prepare_quote(). Both quote() and place_order() now call this method instead of maintaining separate copies of the same code.

**Reason:** Extract Method and Remove Duplicated Code are appropriate because both operations require the same calculations. Keeping the logic in one place reduces the possibility of inconsistent results.

**After:** The methods are shorter and easier to understand. Future changes to validation or pricing can be made in one location instead of two.

**Verification:** The existing quote tests, including test_regular_pickup_quote and test_student_delivery_quote, should be rerun alongside the order-placement tests to confirm that the original calculations remain consistent.

**Refactoring 2: Long Method and Multiple Responsibilities**

**Before:** The original place_order() method performed several tasks, including validation, pricing calculations, creating an order, updating stock, generating an order ID, and saving the updated state.

**Refactoring:** I separated the responsibilities into smaller methods. _prepare_quote() handles validation and pricing, while _update_stock() handles inventory adjustments. The main place_order() method now focuses on creating and recording an order.

**Reason:** Extract Method is appropriate because each operation has a separate responsibility. Smaller methods make the program easier to understand and allow individual operations to be tested more clearly.

**After:** The order-placement process is easier to follow. Validation and pricing happen first, followed by order creation, stock updates, and saving. This reduces the complexity of place_order().

**Verification:** The tests test_order_with_multiple_products, test_order_ids_increase, test_order_can_be_reopened, and test_invalid_order_preserves_previous_order check important order-placement behavior.

**Refactoring 3: Repeated Product Searching**

**Before:** The original implementation repeatedly used loops to search the catalog for matching product IDs. In place_order(), products were searched again when stock was deducted, even though they had already been located during validation.

**Refactoring:** I introduced _product_lookup(), which creates a dictionary mapping each product ID to its product record. The validation and stock-update operations can now access products through this dictionary instead of repeatedly searching the entire catalog.

**Reason:** Replacing repeated searches with a dictionary simplifies the lookup process and reduces unnecessary processing.

**After:** Product lookup is more direct, the code contains fewer nested loops, and inventory operations are easier to maintain.

**Verification:** The tests test_order_with_multiple_products and test_cancel_order_restores_stock help verify that product selection and inventory adjustments continue to work correctly.

**Test Results and Comparison**

**Before refactoring:**

- Original tests: 4 passed.
- Additional tests: 6 passed and 1 failed.
- Total: 11 tests, 10 passed, 1 failed.

**After refactoring:**

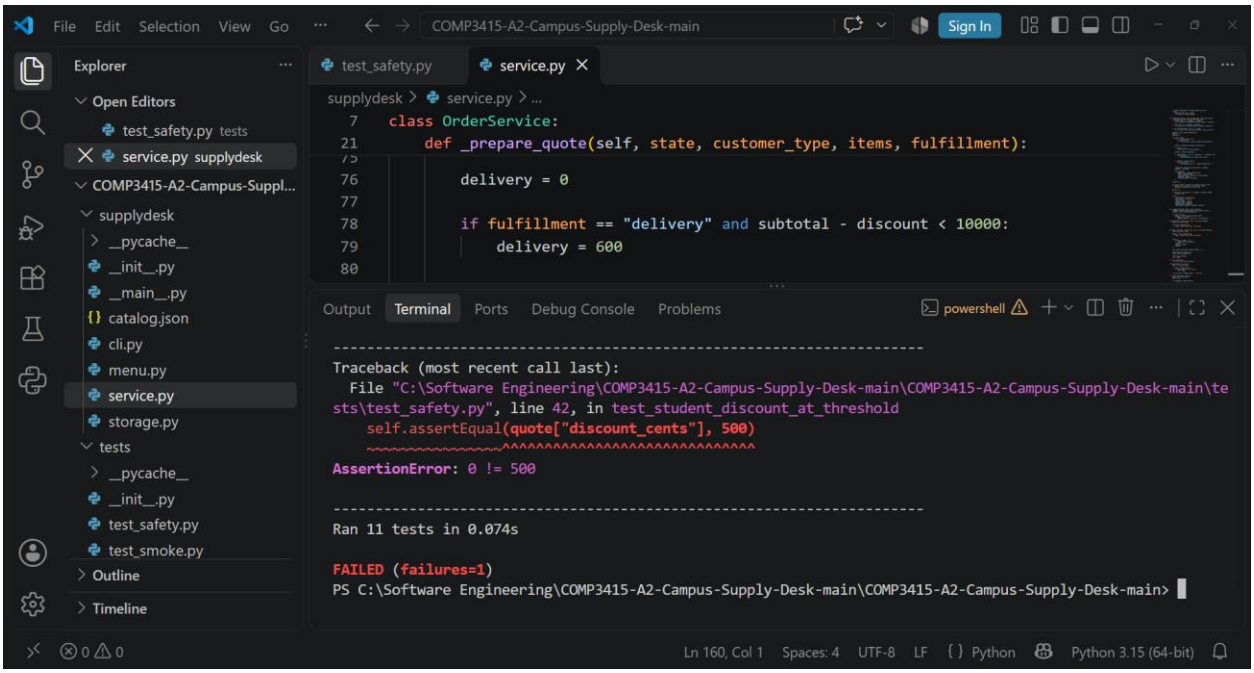
- Tests executed: To be recorded.
- Tests passed: To be recorded.
- Tests failed: To be recorded.

The existing student discount test failed before refactoring because the original implementation did not apply the discount at exactly 5,000 cents. This is a functional defect, not a code smell.

**Separate Functional Defects**

The original implementation also has a stock-validation problem when duplicate product lines exceed available inventory. Additionally, cancel_order() can restore inventory more than once if the same order is cancelled repeatedly.

These behavioral problems should be tested and corrected separately from the three structure-focused refactorings. Their corrections must not be presented as behavior-preserving changes.

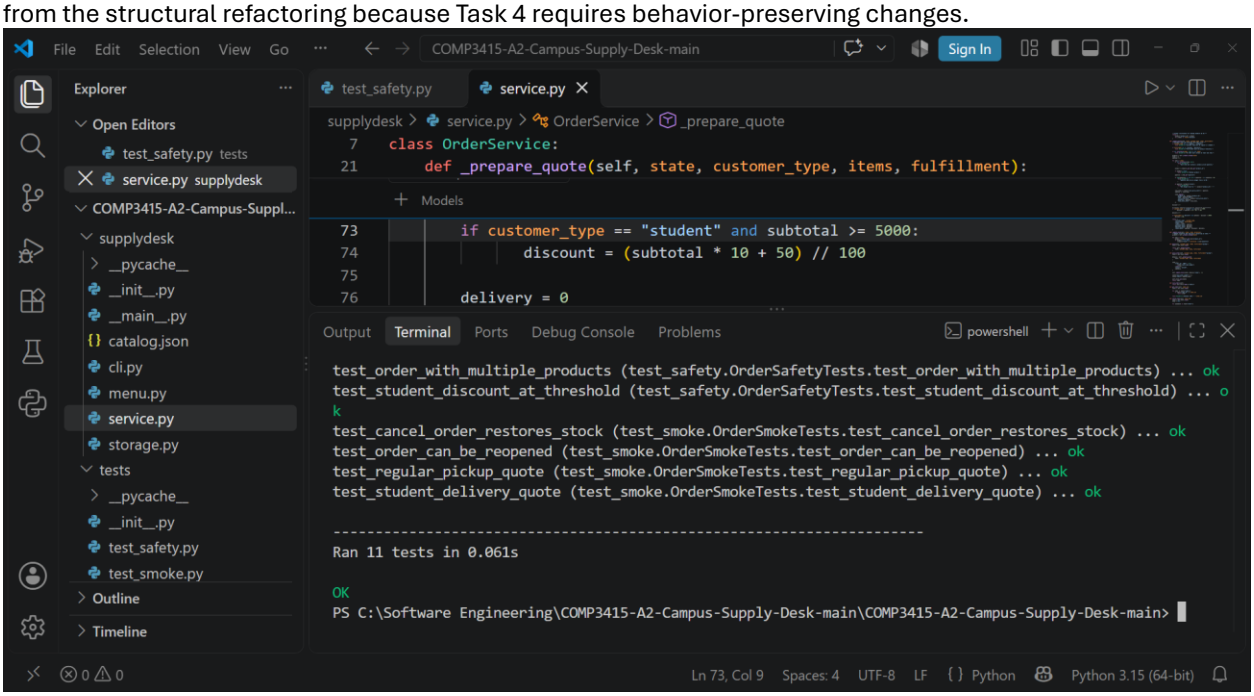



**Verification – Before and After Refactoring**

Before refactoring, I ran 11 automated tests using Python's unittest framework. Ten tests passed, and one failed because the student discount was not applied at exactly $50.

After completing the three refactorings, I ran the same 11 tests again. Ten tests passed, and the same student discount test failed. The results show that the refactoring preserved the existing behavior covered by the tests.

The student discount failure was already present in the original implementation. I kept this defect separate from the structural refactoring because Task 4 requires behavior-preserving changes.



**Task 5**

Part A – Before Implementation

1. Which existing behavior or requirement will change?

The application currently supports two fulfillment methods: pickup and standard delivery. CR-01 requires adding express delivery as a third option.

Express delivery must always cost 1,200 cents ($12), even when the order qualifies for free standard delivery. Student discounts must continue to apply only to merchandise, and the existing pickup and standard delivery rules must remain unchanged.

2. Which files, classes, or methods do I expect to modify?

I expect to make changes to the following components:

- service.py: Update OrderService._prepare_quote() to accept express and calculate the fixed 1,200-cent delivery charge.
- menu.py: Update the interactive menu so customers can select express delivery.
- cli.py: Update the command-line fulfillment options and help information.
- README.md: Document the new express delivery option and provide an example.
- tests/test_safety.py: Add automated tests for express delivery pricing, validation, and order persistence.

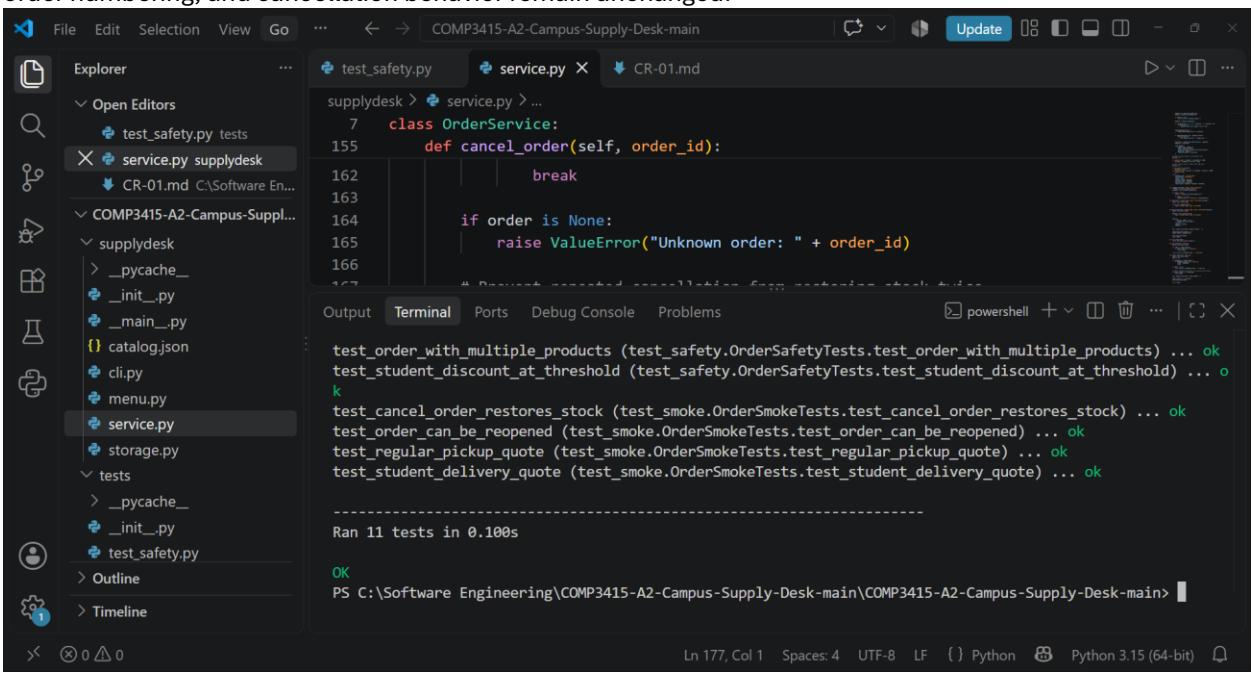
I do not expect major changes to storage.py because fulfillment and delivery charges are already stored in order records.

3. Which existing or new tests will be needed?

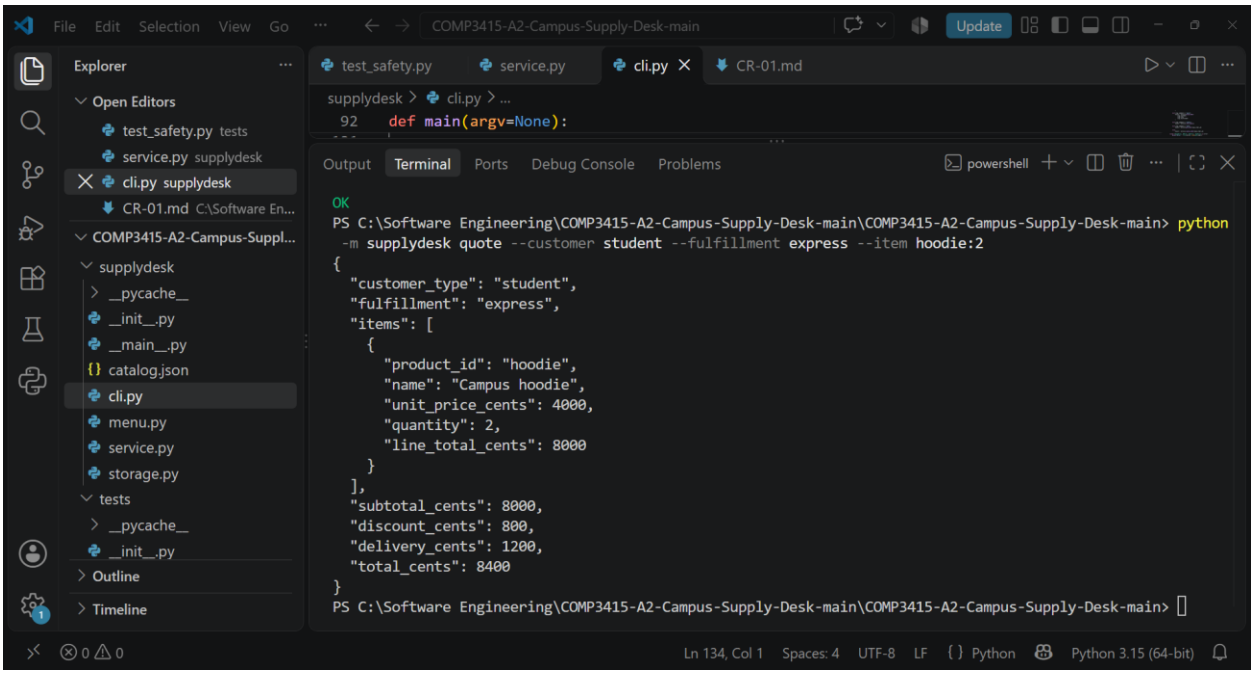
I plan to add tests that verify express delivery charges 1,200 cents for regular and student customers, including orders that qualify for free standard delivery.

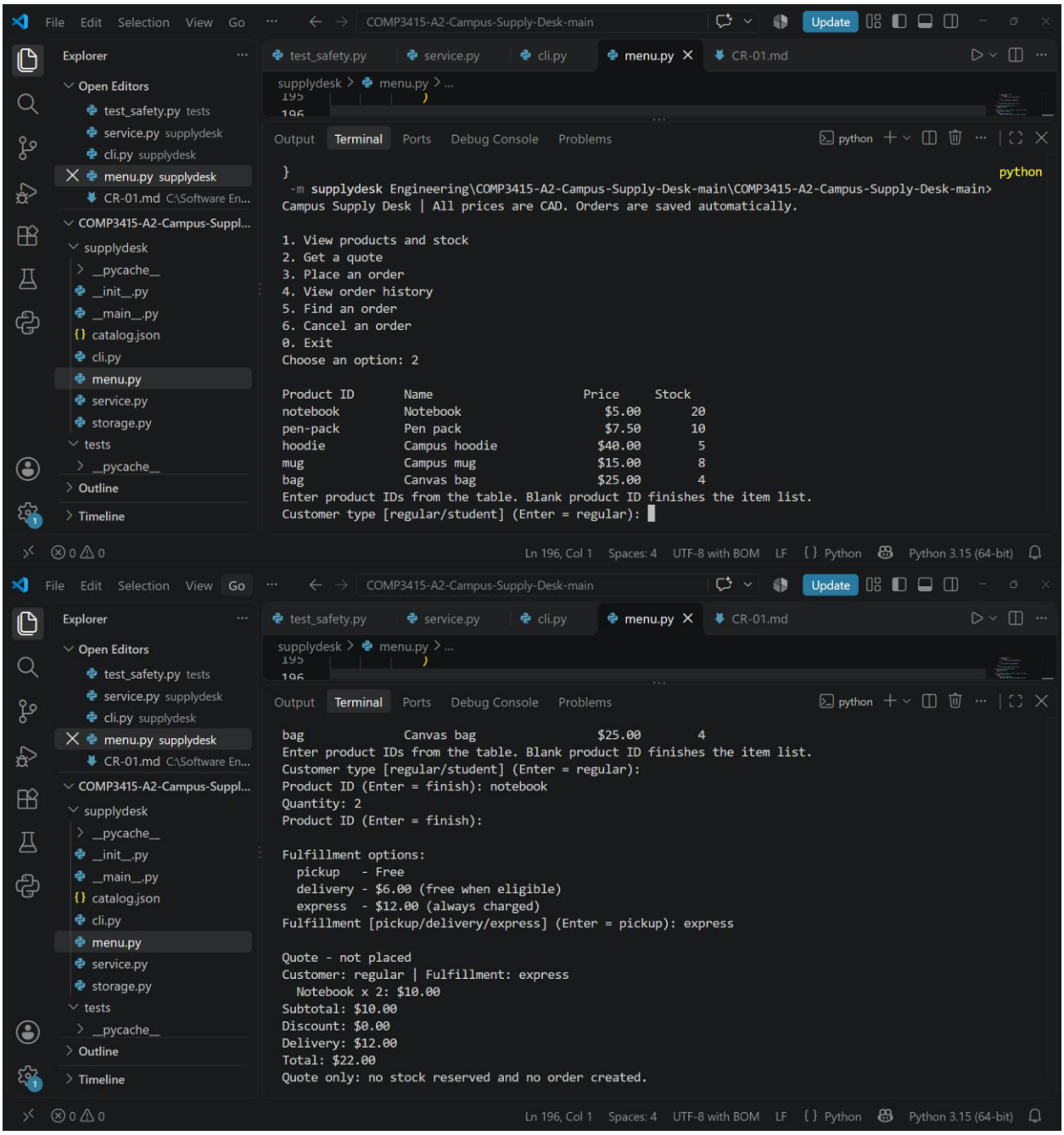
I will also test that quotes and placed orders calculate the same totals, express orders retain their information after reopening or cancellation, and unsupported fulfillment values are still rejected.

Finally, I will rerun the existing 11 tests to confirm that pickup, standard delivery, inventory management, order numbering, and cancellation behavior remain unchanged.




**After CR-01**






Part B – Implementation and Testing

I implemented express delivery in the refactored application. In supplydesk/service.py, the shared _prepare_quote() logic accepts express and applies a fixed 1,200-cent delivery fee. Unlike standard delivery, express is never free. The existing student discount calculation continues to apply to merchandise only.

In supplydesk/cli.py, I added express to the --fulfillment choices and command help. In supplydesk/menu.py, I added express to the fulfillment prompt and displayed its $12.00 fee. The defaults remain regular customer and pickup. No storage format change was needed because order records already contain fulfillment, pricing, and line-item information.

I added five CR-01 tests in tests/test_safety.py: test_express_regular_quote, test_express_student_discount, test_express_fee_not_waived, test_express_order_persistence, and test_invalid_fulfillment_rejected. They cover both customer types, fixed express pricing, the standard-delivery comparison, quote/order agreement, saved-order retrieval, cancellation, and rejection of unsupported fulfillment methods.

Verification: The command-line example for two hoodies with student express delivery returned total_cents = 8400. The interactive menu showed two notebooks with express delivery totaling $22.00. The final automated test screenshot shows 16 tests executed and all 16 passing (11 earlier tests and 5 CR-01 tests). These results support the tested functionality; they do not prove untested inventory edge cases are fixed.

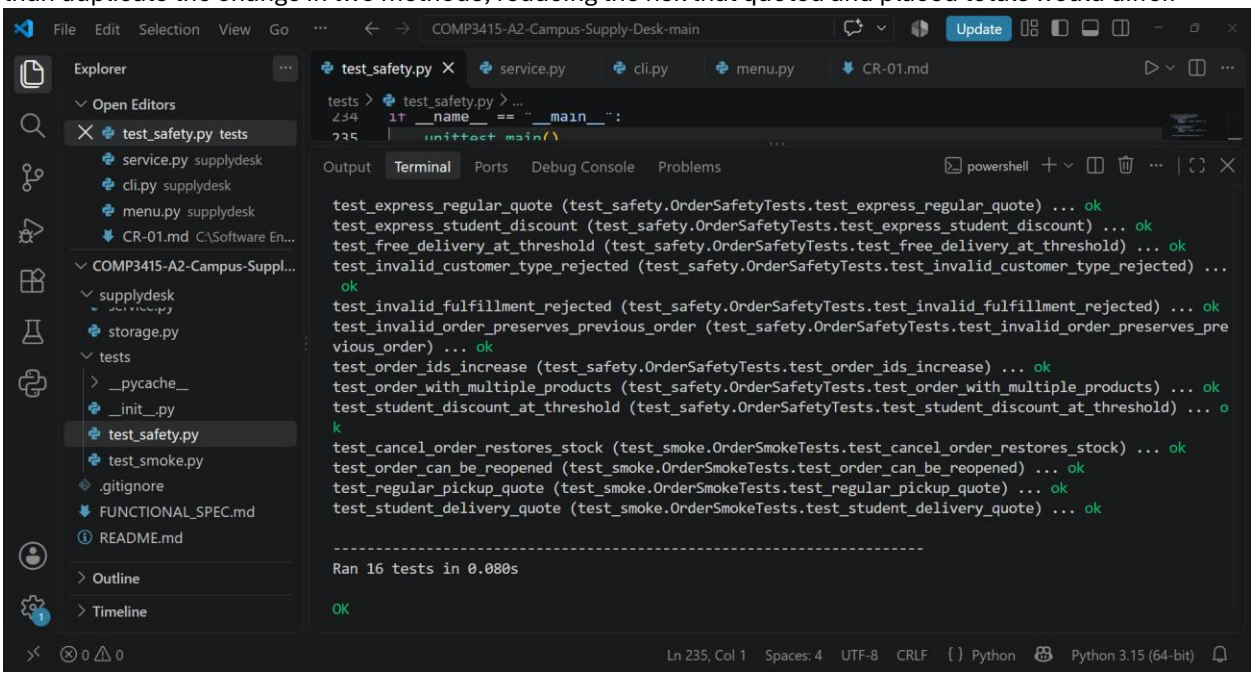
Part C – Review

Actual changes: The implementation involved supplydesk/service.py (_prepare_quote()), supplydesk/cli.py (build_parser()), supplydesk/menu.py (read_request()), and tests/test_safety.py (five additional tests). The README.md update was planned, but the submitted evidence does not confirm that it was saved in the project; it should be checked before final submission.

Prediction compared with implementation: My Part A prediction was largely correct. The service, command-line interface, menu, and tests all needed changes. As expected, storage.py did not require modification. Documentation was also identified as necessary, although its completion still needs confirmation.

Tests added or updated: Five express-specific tests were added. I also reran the existing 11 tests to check that earlier pickup, standard delivery, order numbering, and other tested behavior continued to work. The full suite reported 16 tests and OK.

Effect of refactoring: The earlier extraction of _prepare_quote() made CR-01 easier to implement because quote() and place_order() share the same validation and pricing logic. I could add express pricing once rather than duplicate the change in two methods, reducing the risk that quoted and placed totals would differ.




**Task 6 – AI-Use Record**

**AI Tool:** ChatGPT (OpenAI)

**Purpose of AI Use:**

I used ChatGPT as a learning and support tool to better understand code smells, refactoring techniques, testing strategies, and the requirements of CR-01. AI helped explain programming concepts, suggest possible improvements, and provide example implementations that I could review against the assignment requirements.

**Evaluation of Three AI Recommendations**

**1. Extract Method – ACCEPT**

AI suggested extracting repeated validation and pricing logic into a shared helper method.

I accepted this recommendation because the original quote() and place_order() methods contained duplicated logic. The resulting _prepare_quote() method made the code easier to understand and maintain.

**2. Product Lookup Dictionary – ACCEPT**

AI suggested using a dictionary to locate products by their IDs instead of repeatedly searching through the catalog.

I accepted this recommendation because it simplified product searching and reduced unnecessary loops.

**3. Student Discount Correction – MODIFY**

AI suggested changing the discount condition from subtotal > 5000 to subtotal >= 5000.

I modified how I applied this recommendation: rather than including the threshold change within the behavior-preserving refactoring, I kept the original condition during Task 4 verification and corrected it afterward as a separate functional defect. The baseline test expected a 500-cent discount at exactly $50 but received 0; after changing > to >=, all 11 tests passed. This distinction matters because correcting behavior is not the same as refactoring code structure.

**Testing and Verification**

I used the assignment requirements and automated test results to evaluate the suggested changes.

Before refactoring, 10 out of 11 tests passed. After refactoring, the same 10 tests passed. After correcting the discount condition, all 11 tests passed.

AI also assisted with example code and tests for CR-01 Express Delivery. The final test suite contained 16 passing tests.

**Reflection**

AI was useful for understanding the reasoning behind different programming approaches and identifying possible improvements. I reviewed the suggestions in relation to the supplied code, functional requirements, and testing results.

The assignment documents my implementation process, testing evidence, and evaluation of the recommendations. AI-generated suggestions and examples were used as supporting material rather than treated as automatically correct.